МИНИСТЕРСТВО ЦИФРОВЫХ ТЕХНОЛОГИЙ РЕСПУБЛИКИ УЗБЕКИСТАН
ТАШКЕНТСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ ИМЕНИ МУХАММАДА АЛ-ХОРАЗМИЙ

Дисциплина: «Введение в программную инженерию»

ПРАКТИЧЕСКАЯ РАБОТА № 8

Тема 8. Паттерны проектирования

Вариант № 8
«Разработка умного сайта для заказа услуг»

Выполнила: студентка группы M320-24 ДИР
Камбарова Хатича

Проверила: Латипова Нодира

Ташкент — 2026

1. Тема практической работы

Паттерны проектирования.

Цель работы: закрепить теоретические знания; научиться выявлять в проекте типовые проблемы проектирования, подбирать для них архитектурный паттерн и паттерны GoF, обосновывать выбор по последствиям и применять паттерн в коде.

Вариант № 8: «Разработка умного сайта для заказа услуг» («XizmatGo»). Работа выполнена на основе ПР-5 (требования FR-05 «умный подбор», FR-07 «статус и уведомления», FR-09 «онлайн-оплата»), ПР-6 (подсистемы SS-3 «Заказы», SS-4 «Подбор», SS-5 «Платежи», оценка сцепления и рефакторинг) и ПР-7 (замечания пользователей о рекомендации мастера и способах оплаты).

2. Обоснование выбора архитектурного паттерна

Выбран архитектурный паттерн **«Многоуровневая архитектура» (Layers)**, дополненный **паттерном «Издатель — подписчик» (Publish–Subscribe)** для взаимодействия модулей. Это соответствует стилю «модульный монолит с событиями», выбранному в ПР-6. Уровни: представление (SPA, админ-панель) → прикладной уровень (API-шлюз, контроллеры) → уровень предметной логики (сервисы подсистем SS-1…SS-7) → инфраструктура и доступ к данным (INF-1…INF-5, PostgreSQL). Каждый уровень обращается только к нижележащему через интерфейсы; события (ЗаказСоздан, СтатусИзменён, ОплатаПолучена) публикуются в шину INF-4.

Таблица 2.1 — Сравнение архитектурных паттернов (0–5 баллов)

Критерий (требование)	Layers + Pub–Sub	MVC (только)	Микросервисы	Брокер / SOA
Разделение ответственности, модифицируемость (NFR-08)	5	3	5	4
Слабое сцепление между модулями (итоги ПР-6)	5	2	5	4
Производительность: подбор ≤ 3 с, API ≤ 500 мс (NFR-01)	4	4	3	3
Простота разработки и развёртывания командой из 5 человек	4	5	1	2
Тестируемость (замена уровней заглушками)	5	3	4	3
Итого (из 25)	**23**	**17**	**18**	**16**

Выгоды: каждый уровень можно менять и тестировать отдельно; бизнес-логика не зависит от фреймворка и БД; события убирают прямые зависимости «заказы → уведомления → аналитика». **Издержки:** запрос проходит несколько уровней (небольшие накладные расходы); асинхронные события сложнее отлаживать — нужно журналирование событий. Паттерн MVC (Django MVT) сохраняется внутри уровня представления и прикладного уровня, но сам по себе не решает проблем связей между подсистемами.

3. Описание трёх проектных проблем

Таблица 3.1 — Проблема П-1: меняющиеся правила подбора мастера

Контекст	Подсистема SS-4 «Умный подбор» (UC-04, FR-05) ранжирует свободных мастеров. В ПР-3 формула была одна: рейтинг + расстояние + загрузка. В ПР-7 пользователи и заказчик попросили разное поведение: для заказа «Срочно» важнее всего близость, клиенты вроде П-3 хотят «подешевле», после пилота веса будут уточняться.
Проблема	Как поддерживать несколько алгоритмов ранжирования и менять их без правки основного кода подбора и без разрастания условий `if urgent … elif budget …`?
Силы	• алгоритмы часто меняются, а код подбора (фильтр верифицированных, выдача топ-3) — нет; • каждую формулу нужно отдельно тестировать на эталонных данных; • выбор алгоритма зависит от параметров конкретного заказа во время работы; • в будущем возможен A/B-тест формул и ML-модель; • лишние классы усложняют маленький модуль.

Таблица 3.2 — Проблема П-2: реакция многих модулей на смену статуса заказа

Контекст	При смене статуса заказа (FR-07) нужно: отправить SMS/Telegram клиенту (INF-1), пересчитать рейтинг мастера (SS-6), обновить аналитику (SS-7), при статусе «Завершён» создать счёт на оплату (SS-5). В ПР-6 найдено сцепление по управлению SS-3 → INF-1 и «чужие» обязанности в модуле заказов.
Проблема	Как оповестить все заинтересованные модули об изменении заказа, чтобы класс заказа не знал о них и не менялся при добавлении нового получателя?
Силы	• число получателей растёт (позже — уведомления модератору, push в PWA); • сбой отправки SMS не должен отменять смену статуса; • модуль заказов должен сохранять функциональную связность (ПР-6); • порядок вызова получателей обычно не важен, но важно, чтобы никто не был забыт.

Таблица 3.3 — Проблема П-3: разные интерфейсы платёжных систем

Контекст	Подсистема SS-5 «Платежи» (UC-06, FR-09) работает с Payme, Click и Uzum. У каждого провайдера свой протокол (JSON-RPC или REST), свои названия методов, единицы суммы (сумы или тийины) и статусы. Набор провайдеров может расширяться.

Проблема	Как дать сервису оплаты единый интерфейс «создать платёж / проверить статус», не переписывая его под каждого провайдера и не меняя чужие API?
Силы	• API провайдеров изменить нельзя; • бизнес-логика оплаты (комиссия 10 %, фискальный чек) одинакова для всех; • каждого провайдера нужно тестировать с заглушкой в тестовой среде; • добавление нового провайдера не должно затрагивать существующий код (принцип открытости/закрытости).

4. Таблица «проблема — паттерн — последствия»

Проблема	Паттерн GoF (тип)	Как решает	Выгоды	Издержки
П-1 Меняющиеся правила подбора мастера	**Стратегия (Strategy)** — поведенческий	Каждая формула — отдельный класс с общим интерфейсом RankingStrategy; MatchingService хранит ссылку на стратегию и вызывает score(); стратегия выбирается по флагам заказа	Новая формула — новый класс без правки MatchingService; формулы тестируются отдельно; можно менять стратегию во время работы и проводить A/B-тесты; нет цепочек if-else	Больше классов; клиент должен знать различия стратегий, чтобы выбрать нужную; небольшие накладные расходы на вызов через интерфейс
П-2 Реакция многих модулей на смену статуса	**Наблюдатель (Observer)** — поведенческий	Order — субъект, хранит список OrderObserver и при changeStatus() вызывает onStatusChanged() у каждого; уведомления, рейтинг, аналитика, оплата — конкретные наблюдатели. В системе реализуется через шину событий INF-4	Слабое сцепление «субъект — наблюдатели»; новые получатели подключаются без изменения Order; модуль заказов остаётся функционально связным; сбой одного наблюдателя изолирован	Неочевидный порядок и каскады оповещений; сложнее отлаживать; риск «забытых» подписок и утечек памяти; лишние оповещения, если наблюдателю нужен не каждый статус
П-3 Разные интерфейсы платёжных систем	**Адаптер (Adapter)** — структурный	Единый целевой интерфейс PaymentGateway; для каждого провайдера — адаптер, который переводит вызовы и данные (сумы → тийины, статусы) в формат его API	PaymentService не зависит от конкретных провайдеров; новый провайдер — новый адаптер; адаптеры легко заменить заглушками в тестах; изменения чужого API затрагивают только один класс	Дополнительный уровень и классы; при сильных различиях API адаптер может стать сложным; общий интерфейс ограничивает специфические возможности провайдера

Рассмотренные, но отклонённые альтернативы: для П-1 — *Шаблонный метод* (жёстко фиксирует порядок шагов, менять формулу можно только наследованием, без смены во время работы); для П-2 — *Посредник* (централизует логику в одном классе, который рискует стать «божественным объектом»); для П-3 — *Фасад* (упрощает одну сложную подсистему, но не приводит разные API к одному интерфейсу).

5. Диаграммы классов применения паттернов

На диаграммах участники названы терминами предметной области, а их роли в паттерне показаны стереотипами («Context», «Strategy», «Subject», «Observer», «Target», «Adapter», «Adaptee», «Client»).

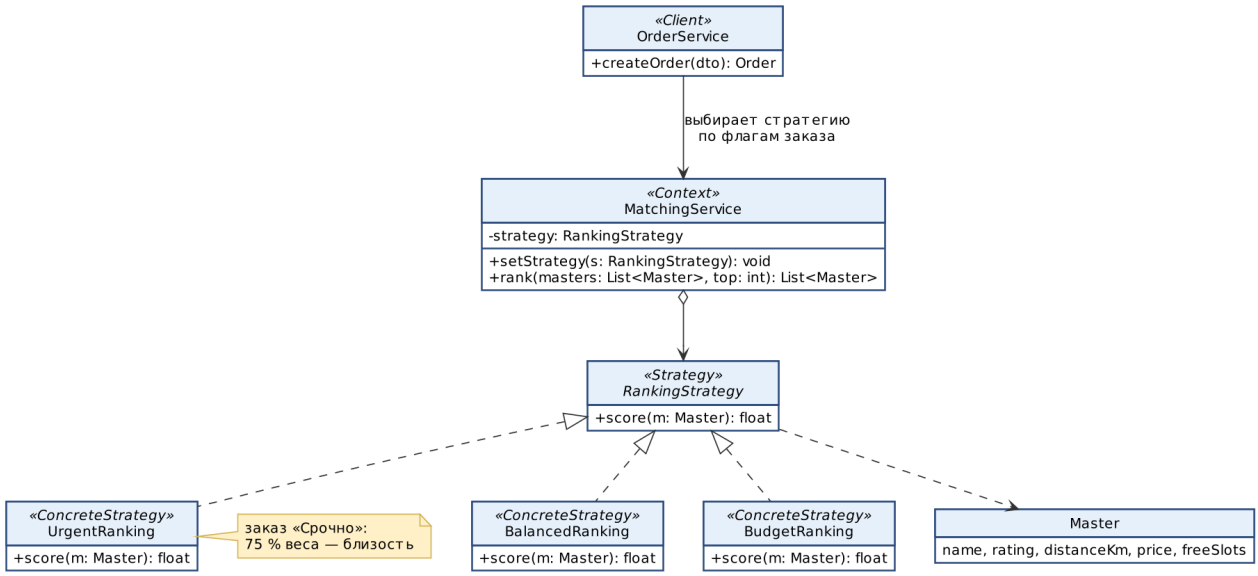


Рисунок 5.1 — Паттерн «Стратегия» в подсистеме SS-4 «Умный подбор»

Роль в паттерне	Класс предметной области
Context	MatchingService — сервис подбора мастеров
Strategy	RankingStrategy — интерфейс «правило ранжирования»
ConcreteStrategy	BalancedRanking (обычный заказ), UrgentRanking (срочный), BudgetRanking (подешевле)
Client	OrderService — выбирает стратегию по признакам заказа

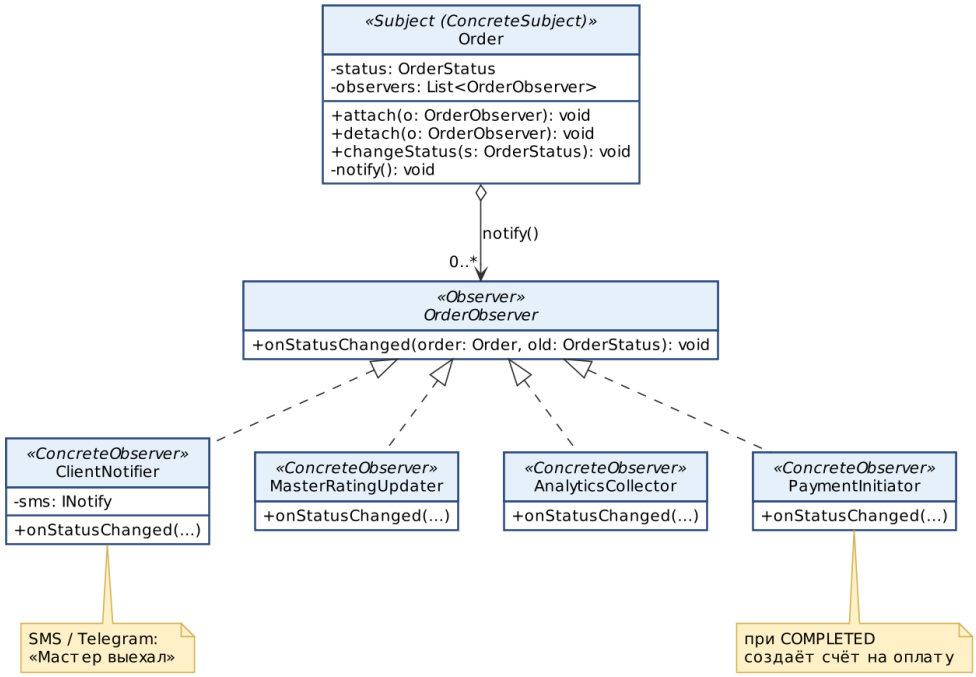


Рисунок 5.2 — Паттерн «Наблюдатель» для смены статуса заказа

Роль в паттерне	Класс предметной области
Subject / ConcreteSubject	Order — заказ, у которого меняется статус
Observer	OrderObserver — интерфейс «получатель изменений заказа»
ConcreteObserver	ClientNotifier (SMS/Telegram), MasterRatingUpdater (рейтинг), AnalyticsCollector (дашборд), PaymentInitiator (счёт после завершения)

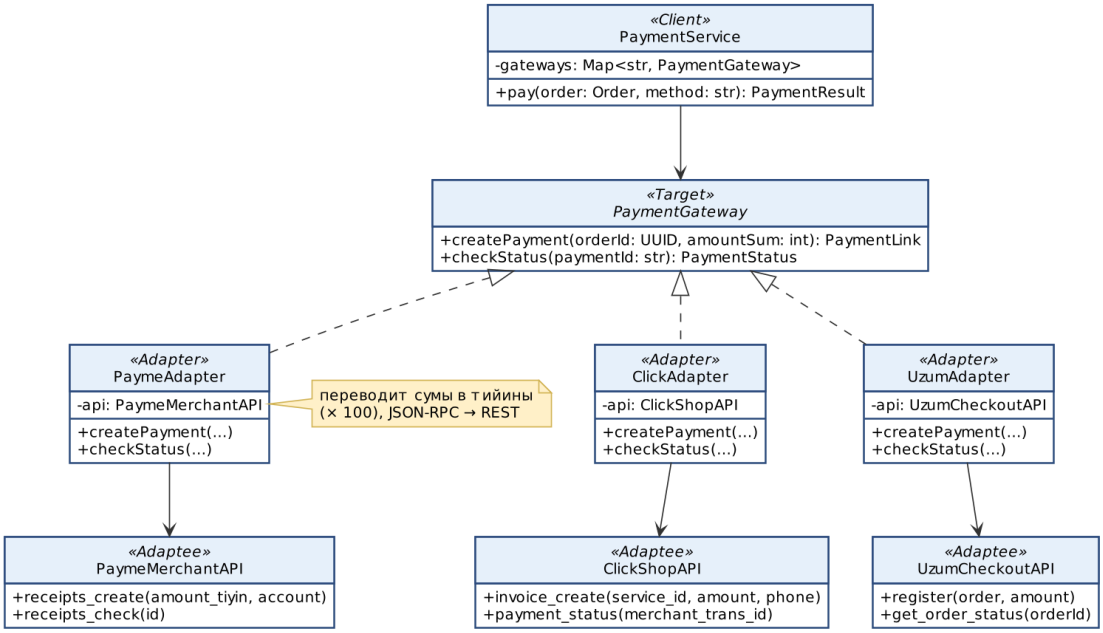


Рисунок 5.3 — Паттерн «Адаптер» в подсистеме SS-5 «Платежи»

Роль в паттерне	Класс предметной области
Client	PaymentService — сервис оплаты заказов
Target	PaymentGateway — единый интерфейс платёжного шлюза
Adapter	PaymeAdapter, ClickAdapter, UzumAdapter
Adaptee	PaymeMerchantAPI, ClickShopAPI, UzumCheckoutAPI — клиенты API провайдеров (названия методов условные, упрощены для учебного примера)

6. Фрагмент кода с пояснениями и результатом выполнения

Реализован паттерн **«Стратегия»** для подбора мастера (Python 3, 55 строк, файл *strategy.py*).

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class Master:                       # участник паттерна: данные контекста
    name: str
    rating: float                   # 1..5
    distance_km: float
    price: int                      # сум
    free_slots_today: int

class RankingStrategy(ABC):         # Strategy — общий интерфейс алгоритмов подбора
    @abstractmethod
    def score(self, m: Master) -> float: ...

class BalancedRanking(RankingStrategy):      # ConcreteStrategy: обычный заказ
    def score(self, m):
        return 0.5 * m.rating / 5 + 0.3 * (1 - min(m.distance_km, 10) / 10) + 0.2 * min(m.free_slots_today, 4) / 4

class UrgentRanking(RankingStrategy):        # ConcreteStrategy: «Срочно» — важнее близость
    def score(self, m):
        return 0.25 * m.rating / 5 + 0.75 * (1 - min(m.distance_km, 10) / 10)

class BudgetRanking(RankingStrategy):        # ConcreteStrategy: «Дешевле» — важнее цена
    def score(self, m):
        return 0.4 * m.rating / 5 + 0.6 * (1 - m.price / 150_000)

class MatchingService:              # Context — подсистема SS-4 «Умный подбор»
    def __init__(self, strategy: RankingStrategy):
        self._strategy = strategy
    def set_strategy(self, strategy: RankingStrategy):
        self._strategy = strategy  # стратегию можно сменить во время работы
    def rank(self, masters, top=3):
        verified = [m for m in masters if m.rating >= 4.0]   # общее правило для всех стратегий
        return sorted(verified, key=self._strategy.score, reverse=True)[:top]

def choose_strategy(urgent: bool, budget: bool) -> RankingStrategy:
    if urgent:
        return UrgentRanking()
    return BudgetRanking() if budget else BalancedRanking()

if __name__ == "__main__":
    masters = [
        Master("Алишер Т.", 4.9, 1.8, 95_000, 2),
        Master("Бахром Р.", 4.8, 3.2, 90_000, 4),
        Master("Сардор К.", 4.7, 0.6, 110_000, 1),
        Master("Жасур М.", 4.5, 5.5, 70_000, 3),
        Master("Отабек Н.", 3.6, 0.9, 60_000, 4),   # рейтинг < 4 — не попадёт в выдачу
    ]
    service = MatchingService(BalancedRanking())
    for title, urgent, budget in [("Обычный заказ", False, False), ("Срочно (+30 %)", True, False), ("Подешевле", False, True)]:
        service.set_strategy(choose_strategy(urgent, budget))
        print(f"{title} — {type(service._strategy).__name__}:")
        for i, m in enumerate(service.rank(masters), 1):
            print(f"  {i}. {m.name:<10} ★{m.rating}  {m.distance_km} км  {m.price:,} сум  score={service._strategy.score(m):.3f}".rep
```

Пояснения:

- **Строки 4–10** — класс данных Master (имя, рейтинг, расстояние, цена, свободные слоты).
- **Строки 12–14** — абстрактная стратегия RankingStrategy с единственным методом score().
- **Строки 16–26** — три конкретные стратегии с разными весами: обычная (рейтинг 50 %, близость 30 %, свободность 20 %), срочная (близость 75 %), экономная (цена 60 %).
- **Строки 28–35** — контекст MatchingService: общее для всех стратегий правило (в выдачу попадают только мастера с рейтингом ≥ 4,0) и сортировка по оценке текущей стратегии; set_strategy() меняет стратегию во время работы.
- **Строки 37–40** — выбор стратегии по признакам заказа (в системе это делает OrderService).
- **Строки 42–55** — демонстрация: один и тот же список из 5 мастеров ранжируется тремя стратегиями.

Результат выполнения:

```
$ python3 strategy.py
Обычный заказ — BalancedRanking:
  1. Бахром Р.  ★4.8  3.2 км  90 000 сум  score=0.884
  2. Алишер Т.  ★4.9  1.8 км  95 000 сум  score=0.836
  3. Сардор К.  ★4.7  0.6 км  110 000 сум  score=0.802
Срочно (+30 %) — UrgentRanking:
  1. Сардор К.  ★4.7  0.6 км  110 000 сум  score=0.940
  2. Алишер Т.  ★4.9  1.8 км  95 000 сум  score=0.860
  3. Бахром Р.  ★4.8  3.2 км  90 000 сум  score=0.750
Подешевле — BudgetRanking:
  1. Жасур М.   ★4.5  5.5 км  70 000 сум  score=0.680
  2. Бахром Р.  ★4.8  3.2 км  90 000 сум  score=0.624
  3. Алишер Т.  ★4.9  1.8 км  95 000 сум  score=0.612
```

Анализ результата. Одни и те же данные дают разный порядок без изменения кода MatchingService: в обычном режиме первым идёт свободный Бахром Р., для срочного заказа — ближайший Сардор К. (0,6 км), в режиме «подешевле» — Жасур М. (70 000 сум). Мастер Отабек Н. с рейтингом 3,6 не попал ни в одну выдачу, потому что общее правило фильтра находится в контексте, а не в стратегиях. Чтобы добавить, например, стратегию «Любимый мастер клиента», достаточно написать новый класс с методом score().

7. Возможный антипаттерн и способ его исправления

Таблица 7.1 — Антипаттерн «Божественный объект» (God Object / Blob)

Как мог возникнуть	Без выбранных паттернов вся логика заказа оказалась бы в одном классе OrderService: подбор мастера с формулами для каждого режима, рассылка SMS и Telegram, пересчёт рейтинга, обновление аналитики, вызовы Payme, Click и Uzum с их форматами.
Симптомы	• класс на 1 500+ строк и 40+ методов, большинство из которых не связаны между собой; • длинные цепочки `if method == "payme" … elif "click" …` и `if urgent … elif budget …`; • зависимость от всех подсистем и внешних API сразу — сцепление по содержимому и общее (как в оценке «до» ПР-6); • любое изменение (новая формула, новый провайдер) требует правки этого класса и ломает соседние функции; • класс невозможно протестировать без БД, SMS-шлюза и платёжных систем; постоянные конфликты слияния в Git.
Способ исправления	Рефакторинг «Выделение класса» и замена условной логики полиморфизмом: формулы подбора → классы Стратегии; рассылка, рейтинг, аналитика → Наблюдатели (события шины INF-4); провайдеры оплаты → Адаптеры с общим интерфейсом PaymentGateway. В OrderService остаются только сценарии работы с заказом (создать, принять, сменить статус) — функциональная связность. Рефакторинг выполняется небольшими шагами под защитой автотестов.

Антипаттерн при неуместном применении — «Золотой молоток» (Golden Hammer). Симптомы: команда, освоившая Наблюдатель и Стратегию, начинает применять их везде, например делает «стратегию» для единственной формулы расчёта выезда или подписывает на события простые синхронные проверки. Код становится запутанным, а логика — неочевидной («кто изменил статус?»). Исправление: применять паттерн только тогда, когда есть реальная изменчивость или несколько получателей (правило «трёх случаев»), и обсуждать необходимость паттерна на код-ревью.

8. Вывод о проделанной работе

В практической работе по варианту № 8 «Разработка умного сайта для заказа услуг» применены паттерны проектирования к системе «XizmatGo»:

1. Выбран архитектурный паттерн «Многоуровневая архитектура» с «Издатель — подписчик». Он набрал 23 балла из 25 в сравнении с MVC, микросервисами и брокером и согласуется со стилем, выбранным в ПР-6.
2. На основе ПР-5–7 выявлены три проектные проблемы, каждая описана в форме «контекст — проблема — силы»: меняющиеся правила подбора мастера, реакция многих модулей на смену статуса заказа, разные интерфейсы платёжных систем.
3. Для них выбраны паттерны GoF Стратегия, Наблюдатель и Адаптер. В таблице «проблема — паттерн — последствия» указаны выгоды и издержки каждого решения, а также отклонённые альтернативы.
4. Построены 3 диаграммы классов, где участники названы терминами предметной области, а их роли обозначены стереотипами.
5. На Python реализован паттерн Стратегия (55 строк). Запуск показал, что смена стратегии меняет порядок мастеров без изменения кода сервиса подбора.
6. Описаны антипаттерны «Божественный объект» (без паттернов) и «Золотой молоток» (при неуместном применении), их симптомы и способы исправления.

Применение паттернов делает систему открытой для изменений, которые ожидаются после пилотного запуска: новые формулы подбора, получатели уведомлений и платёжные провайдеры добавляются новыми классами без правки существующего кода.